

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе

по курсу «Программирование на языке Java»

на тему «Разработка многомодульного приложения на языке Java»

22.05.25
отлично
Зубри

Выполнил:

студент группы 22ВВП1:

Зубрикова А.А.

Принял:

к.т.н., доцент Юрова О.В.

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет Вычислительной техники
Кафедра "Вычислительная техника"

"УТВЕРЖДАЮ"

Зав. кафедрой ВТ

 профессор М. А. Митрохин

« ____ » _____ 20__ г

ЗАДАНИЕ

на курсовое проектирование по курсу

Программирование на языке Java
Студенту Зубриной Арины Аскольдовне Группа 22 ВВП1
Тема проекта компьютерная игра "Морской бой"

Исходные данные (технические требования) на проектирование

Разработать игровое приложение "Морской бой".
Язык программирования Java, среда разработки NetBeans
Приложение должно обладать графическим интерфейсом
и использовать следующие технологии:

1. Java Collections Framework;
2. Механизм обработки исключительных ситуаций;
3. Java Stream API;
4. Java Multithreading;
5. Сетевое взаимодействие.

Объем работы по курсу

1. Расчетная часть

2. Графическая часть

Диаграммы языка UML: диаграмма вариантов использования;
диаграмма классов; диаграмма деятельности; диаграмма
развертывания; диаграмма последовательности

3. Экспериментальная часть

Тестирование многомодульного приложения

Срок выполнения проекта по разделам

1 В соответствии с графиком

2

3

4

5

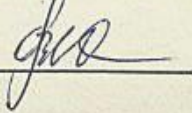
6

7

8

Дата выдачи задания " 19 " 02 . 2025

Дата защиты проекта " ____ " ____

Руководитель Юрова О.В. 

Задание получил " 19 " 02 2025г.

Студент Зубрикова А.А. 

Содержание

Введение	6
1 Постановка задачи.....	7
2 Выбор решения.....	8
3 Описание программы	9
3.1 Сервер	9
3.2 Клиент	10
4 Описание работы программы	14
Заключение	18
Список литературы	19
Приложение А. Листинг программы.....	20
Приложение А.1 Класс SeaBoy. Клиент.	20
Приложение А.2 Класс Client. Клиент.	23
Приложение А.3 Класс Partner. Клиент.	25
Приложение А.4 Класс Game. Клиент.	27
Приложение А.5 Класс Field. Клиент.	33
Приложение А.6 Класс Place. Клиент.	34
Приложение А.7 Cell. Клиент.....	36
Приложение А.8 Класс SeaServer. Сервер.....	37
Приложение А.9 Класс Client. Сервер.	38
Приложение А.10 Класс Server. Сервер.....	39
Приложение Б. UML – диаграммы.....	40
Приложение Б.1 UML – диаграмма последовательности	40
Приложение Б.2 UML – диаграммы классов	41

Приложение Б.3 UML – диаграмма деятельности	43
Приложение Б.4 UML – диаграмма вариантов использования	44
Приложение Б.5 UML – диаграмма развертывания	45

Введение

В рамках данной курсовой работы разрабатывается многомодульное приложение на языке Java с использованием среды разработки NetBeans. Программа включает графический интерфейс и реализует основные возможности Java, включая обработку исключительных ситуаций, многопоточность, а также организацию клиент-серверного взаимодействия.

Актуальность проекта связана с необходимостью практического освоения современных средств Java, востребованных при создании сетевых приложений. Модульная структура проекта повышает читаемость кода, а также способствует гибкости и расширяемости системы. Внедрение клиент-серверной архитектуры демонстрирует основы обмена данными между пользователями приложения. Графический интерфейс позволяет сделать взаимодействие с программой интуитивно понятным и наглядным.

1 Постановка задачи

Требуется разработать игровое приложение, представляющее собой систему программ клиент-серверной архитектуры, реализующую взаимодействие пользователей на основе обмена сообщениями в локальной сети.

Клиентская часть содержит графический оконный интерфейс пользователя, сервер реализуется в виде консольного приложения.

Приложение должно использовать следующие технологии:

- Java Collections Framework;
- Механизм обработки исключительных ситуаций;
- Java Stream API;
- Java Multithreading;
- Сетевое взаимодействие.

Необходимо реализовать следующие функции:

- Организация взаимодействия между клиентами;
- Инициализация игрового поля и возможность расстановки кораблей перед началом сражения;
- Передача ходов между игроками и отображение текущего состояния игры;
- Обработка выстрелов: определение попадания, промаха или уничтожения корабля;
- Отображение визуального состояния полей игрока и противника;
- Завершение игры при уничтожении всех кораблей одного из участников;
- Вывод на экран сообщения о победе или поражении.

Диаграмма деятельности приложения, демонстрирующая все представленные функции приложения, представлена в приложении Б.3.

2 Выбор решения

Для разработки сетевой игры «Морской бой» было принято решение использовать язык программирования Java, так как он предоставляет множество инструментов для построения многомодульных, сетевых и графических приложений. Язык обладает высоким уровнем абстракции, строгой типизацией, большой стандартной библиотекой и возможностями работы с потоками, событиями и сетевыми соединениями. Одним из важнейших преимуществ Java является наличие встроенной поддержки для работы с TCP-соединениями, что важно для реализации клиент-серверного взаимодействия.

В качестве среды разработки выбрана NetBeans IDE, поскольку она обеспечивает удобное управление проектами, интеграцию с Java Swing для разработки графического интерфейса, а также мощные средства для отладки и тестирования. Использование данной среды позволило упростить организацию многомодульной архитектуры приложения.

Обмен сообщениями между клиентами осуществляется с использованием протокола TCP/IP. Такой подход обеспечивает надежную передачу данных, гарантию доставки и упорядоченность пакетов, что важно для синхронизации игры между игроками.

Пользовательский интерфейс реализован с использованием библиотеки Swing, которая предоставляет широкие возможности для построения графических компонентов: кнопок, панелей, диалоговых. Swing позволяет гибко управлять расположением и состоянием компонентов, а также обновлять интерфейс в зависимости от логики игры.

3 Описание программы

3.1 Сервер

Диаграмма последовательности приложения, демонстрирующая взаимодействие клиента и сервера, представлена в приложении Б.1.

Диаграмма классов программы-сервера показана в приложении Б.2.

1) **Класс SeaServer.** Содержит метод `main`, с которого начинается выполнение серверной части игры. Класс запускает сервер в отдельном потоке и завершает программу, если пользователь нажмет кнопку `Enter`.

2) **Класс Server** представляет серверную часть игры.

Сервер слушает порт 60111 и ожидает подключения клиентов в бесконечном цикле. Могут подключиться только два клиента. Для каждого подключенного клиента сервер создает отдельный поток.

Сервер пересылает данные между двумя клиентами с помощью функции `onRead`.

3) **Класс Client** обеспечивает приём и пересылку данных через сокет. С помощью функции `read` читает данные из входного потока сокета и передает сообщение другому клиенту, вызывая функцию сервера `onRead`. Функция `write` вызывается на сервере и передает данные текущему клиенту.

3.2 Клиент

Диаграмма классов программы-клиента показана в приложении Б.2.

1) Файл **SeaBoy.java** содержит функцию `main`. Данная функция создает объекты классов `Client` (в отдельной нити), `Partner`, `Game` и обеспечивает взаимодействие между ними. Затем создается объект класса `SeaBoy`, устанавливается связь между объектами классов `Game` и `SeaBoy` для отображения состояния игры в интерфейсе.

Класс `SeaBoy` представляет собой графическую часть приложения. Создает главное окно приложения, панели, кнопки, слушатели и `action command` для кнопок. Создает объекты класса `Field` – поле игрока и поле противника, отображая их в интерфейсе.

Функция `onStateStarted` – переключает доступность кнопок в зависимости от состояния игры.

Функция `setStatusString` – отображает текст в нижней панели приложения, описывающий текущее состояние игры.

Функция `setFieldCell` – обновляет иконку конкретной клетки на соответствующем игровом поле.

2) **Класс `Field`** создает игровое поле – сетку из кнопок 10 на 10. Устанавливает для каждой кнопки иконку, слушателя и `action command`.

Функция `setCell` устанавливает соответствующую иконку кнопки в зависимости от переданного состояния клетки (`Cell: EMPTY, SHIP, MISS, HIT, KILL`).

Файл **`Cell.java`** содержит перечисление **`Cell`**, определяющее возможные состояния ячейки на игровом поле: `EMPTY` – пустая ячейка, `SHIP` – ячейка с кораблём, `MISS` – промах, `HIT` – попадание по кораблю, `KILL` – убитый корабль.

3) **Класс `Client`** отвечает за установку соединения с сервером, чтение и отправку сообщений. Клиент подключается к серверу по определенному адресу и порту.

Функция `readline` ожидает входящие строки сообщений и пересылает их в объект класса `Partner`.

Класс содержит функцию `write(String s)`, разбирающую полученное сообщение на байты и вызывающую функцию `write(byte[] b, int len)`, которая пишет в сокет полученный байтовый массив.

При разрыве соединения клиент пытается переподключиться 1 раз в секунду.

4) **Класс `Partner`** отвечает за сетевое взаимодействие между двумя игроками и обработку игровых команд, получаемых по сети.

Функция `onReceive` разбирает пришедшую команду и вызывает соответствующий метод у объекта `Game`. Обрабатывает команды: `START`, `FINISH`, `SHOT x y`, `MISS x y`, `HIT x y`, `KILL x y`. Отправляет команды другому игроку с помощью методов: `sendStart()`, `sendFinish()`, `sendShot(x, y)`, `sendMiss(x, y)`, `sendHit(x, y)`, `sendKill(x, y)`.

5) **Класс `Game`** реализует основную игровую логику для сетевой игры «Морской бой». Класс обрабатывает действия игрока и события, приходящие от противника, управляет состоянием поля и взаимодействует с пользовательским интерфейсом.

В конструкторе класса выполняется очистка поля противника и расстановка кораблей на поле игрока.

Функция `setSeaBoy` отвечает за обновление состояния поля игры при ее запуске.

Метод `actionPerformed` обрабатывает события от кнопок поля. В зависимости от нажатой игроком кнопки он вызывает соответствующую функцию: `onButtonStart`, `onButtonPlace`, `onButtonFinish`, `onButtonAbout`, `onButtonShot`, `onButtonMy`.

Функция `onButtonStart` вызывается при нажатии игроком на кнопку «Начать игру». Если противник уже прислал игроку запрос на начало игры, то игра начинается, иначе игрок шлет противнику запрос.

Функция `onButtonPlace` вызывается при нажатии игроком на кнопку «Разместить». Создается объект класса `Place`, выполняющий расстановку кораблей.

Функция `onButtonFinish` вызывается при нажатии игроком на кнопку «Завершить игру». Противнику отсылается сообщение об окончании игры, игра завершается.

Функция `onButtonAbout` вызывается при нажатии игроком на кнопку «О программе». Появляется диалоговое окно с информацией о создателе игры и его руководителе.

Функция `onButtonShot` вызывается при нажатии игроком на кнопку поля противника. Если игрок нажимает на неоткрытую кнопку поля противника, то последнему отсылается сообщение с координатами выстрела. Если игрок нажимает на уже открытую ячейку, то выводится сообщение в строку состояния в нижней панели окна игры.

Функция `onButtonMy` вызывается при нажатии игроком на кнопку своего поля. В зависимости от состояния нажатой ячейки выводится надпись в строку состояния.

Функция `setStateStarted` вызывается для изменения статуса игры: начало игры/завершение игры.

Функция `updateField` вызывается при очищении поля противника или при новой расстановке кораблей игроком, чтобы изменения отобразились в интерфейсе пользователя.

Функция `clearCellPartner` устанавливает все ячейки поля противника как пустые (`EMPTY`).

Функция `onReceiveStart` вызывается, если партнер отправил сообщение "start". Если игрок уже отправлял сообщение о начале игры, то игра начинается, иначе в строке состояния появляется сообщение "Ваш друг хочет начать игру".

Функция `onReceiveFinish` вызывается, если партнер отправил сообщение "finish". Появляется надпись в строке состояния об окончании игры, а также диалоговое окно об окончании игры.

Функция `onReceiveShot` вызывается, если партнер отправил сообщение "shot x y". Если партнер промахнулся, то ему отправляется сообщение о промахе, меняется состояние ячейки поля игрока, ход переходит игроку. Если партнер попал, ему отправляется сообщение о попадении, меняется состояние ячейки поля игрока. Затем проверяется, не убит ли корабль целиком в функции `isKilled`. Если корабль убит, то функция `setKilled` меняет все ячейки убитого корабля на поле игрока, а также отправляет сообщение, чтобы на поле противника корабль также был отмечен как убитый. Затем проверяется не завершена ли игра. Это выполняется в функции `isOver`, которая проверяет все ячейки на поле игрока. Если на поле 20 ячеек со статусом KILL, игра завершается – игрок проиграл. После чего появляется диалоговое окно о поражении.

Функция `onReceiveMiss` вызывается, если партнер отправил сообщение "miss x y". Т.е выстрел, который совершил игрок является промахом. Состояние ячейки поля, в которую стрелял игрок, меняется на MISS.

Функция `onReceiveHit` вызывается, если партнер отправил сообщение "hit x y". Т.е выстрел, который совершил игрок является попадением. Состояние ячейки поля, в которую стрелял игрок, меняется на HIT.

Функция `onReceiveKill` вызывается, если партнер отправил сообщение "kill x y". Т.е выстрел, который совершил игрок, уничтожил корабль. Состояние ячейки поля, в которую стрелял игрок, меняется на KILL. Затем проверяется, не окончена ли игра. Это выполняется в функции `isOver`, которая проверяет все ячейки на поле противника. Если на поле 20 ячеек со статусом KILL, игра завершается – игрок выиграл. После чего появляется диалоговое окно о выигрыше.

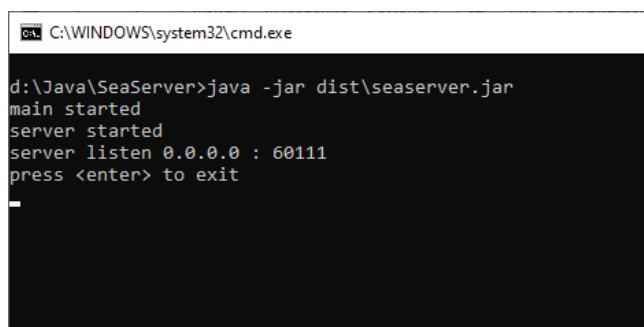
6) **Класс Place** содержит функцию `ship`, которая выполняет автоматическую расстановку кораблей на поле.

4 Описание работы программы

Диаграмма развертывания приложения, демонстрирующая все компоненты приложения, представлена в приложении Б.5.

Ниже представлена работа клиент-серверного приложения «Морской бой».

Запускается сервер:



```
C:\WINDOWS\system32\cmd.exe

d:\Java\SeaServer>java -jar dist\seaserver.jar
main started
server started
server listen 0.0.0.0 : 60111
press <enter> to exit
```

Рисунок 1 – Запуск сервера

Запускаются клиенты:

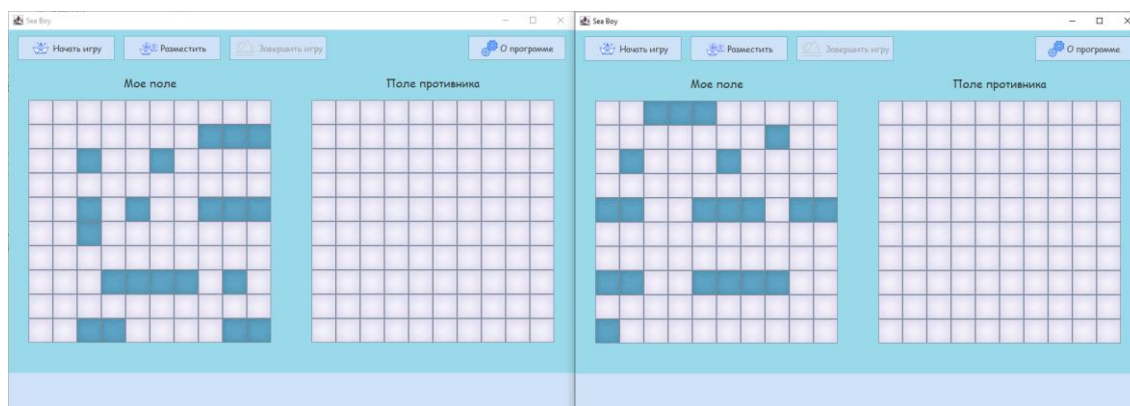


Рисунок 2 – Запуск клиентов

При запуске приложения-клиента выполняется автоматическое размещение кораблей на поле игрока. Можно нажать на кнопку «Разместить», чтобы изменить положение кораблей.

Один из игроков нажимает кнопку «Начать игру». После этого у соперника в строке состояния (нижняя панель приложения) появляется текст: «Ваш друг хочет начать игру».

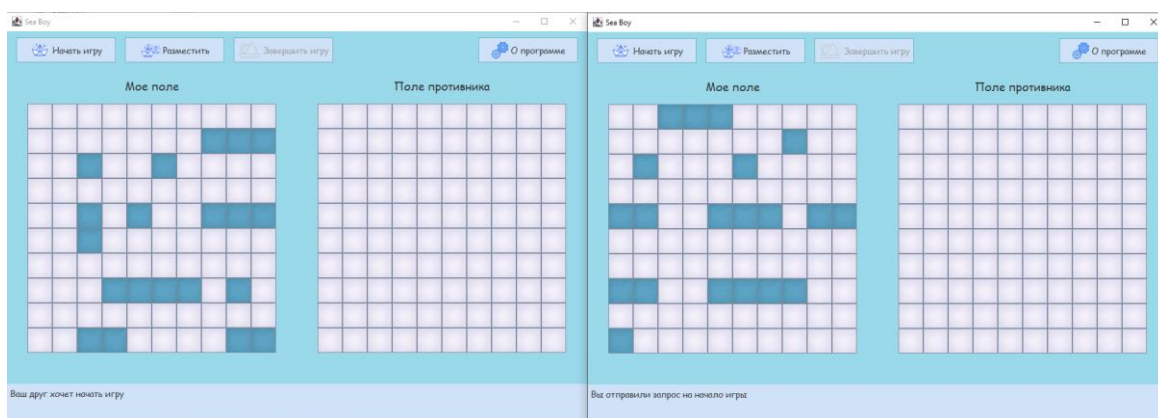


Рисунок 3 – Отправка запроса о начале игры

Когда соперник также нажимает на кнопку «Начать игру», игра начинается. В строке состояния появляется информация о начале игры и об очередности хода. Кнопки «Начать игру» и «Разместить» теперь недоступны, а кнопка «Завершить игру» становится доступной.

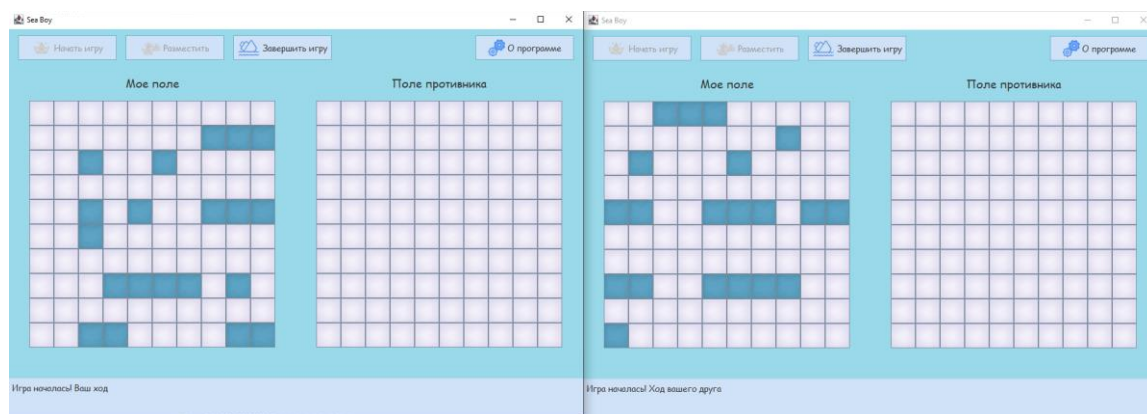


Рисунок 4 – Игра началась

Первым ходит тот пользователь, который нажал кнопку «Начать игру» вторым. Если попасть на ячейку, где нет корабля, ее иконка заменяется иконкой с точкой. При попадении в корабль иконка этой клетки заменяется на иконку со светлым крестиком. Если корабль полностью убит, во всех его ячейках устанавливаются иконки с красными крестиками.

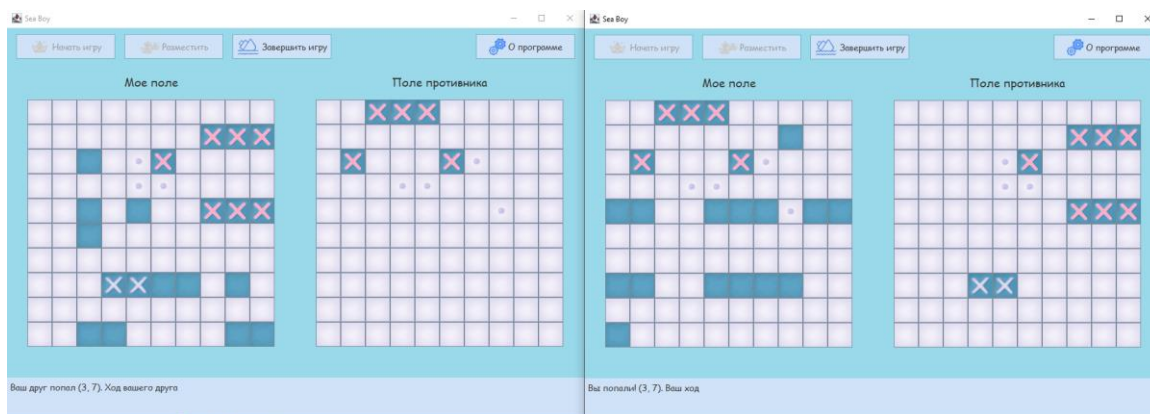


Рисунок 5 – Ход игры

При уничтожении одним из игроков всех кораблей противника игра завершается. У победителя в строке состояния появляется: «Вы победили!», а диалоговое окно говорит о выигрыше. У проигравшего в строке состояния появляется: «Вы проиграли!», а диалоговое окно говорит о проигрыше. У каждого пользователя поле противника очищается, а собственное поле остается нетронутым, пока игрок не нажмет кнопку «Разместить корабли» или пока не начнется новая игра.

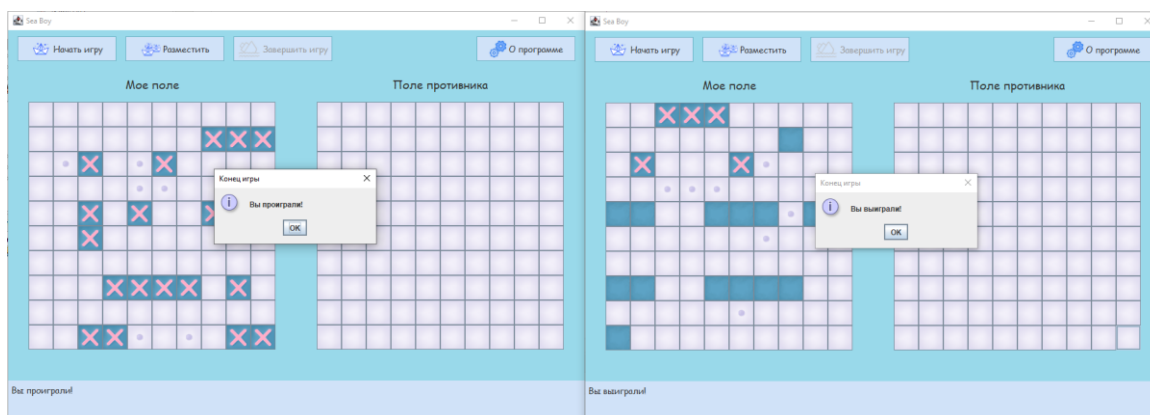


Рисунок 6 – Окончание игры

При нажатии на кнопку «Завершить игру» можно досрочно закончить игру.

При нажатии на кнопку «О программе» появляется диалоговое окно, содержащее информацию о создателе игры и о руководителе проекта:

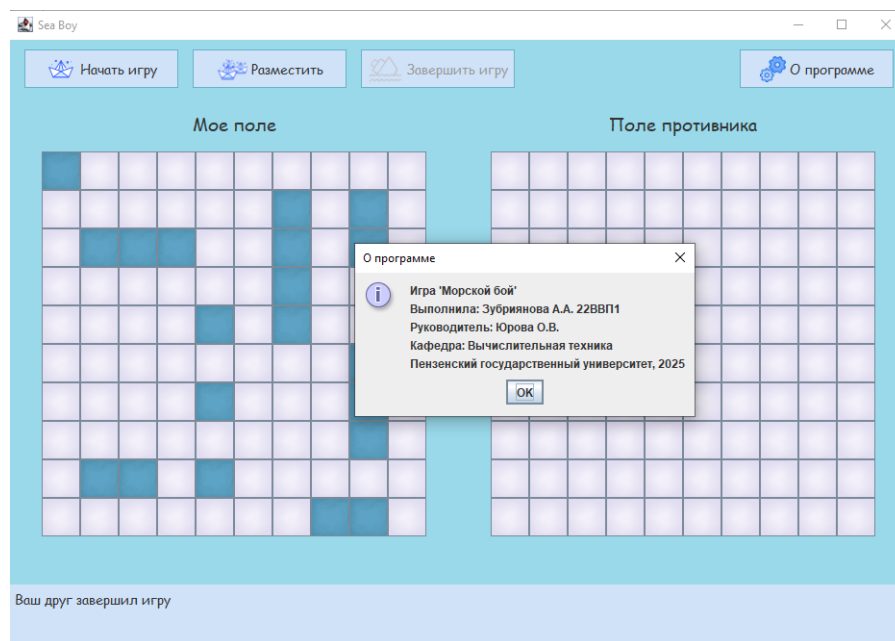


Рисунок 7 – О программе

Для завершения работы сервера нужно нажать кнопку Enter.

Диаграмма вариантов использования приложения, демонстрирующая способы взаимодействия пользователя с системой, представлена в приложении Б.1.

Заключение

В ходе выполнения курсовой работы была разработана сетевая игра «Морской бой» с графическим интерфейсом пользователя на языке Java. Приложение позволяет двум пользователям подключаться друг к другу через сеть и играть в классическую версию игры, реализуя основные функции: расстановка кораблей, чередование ходов, проверка попаданий и поражений, а также вывод сообщений о ходе игры и её завершении.

В процессе разработки были изучены и применены ключевые технологии Java, такие как работа с графическим интерфейсом с помощью библиотеки Swing, обработка событий, взаимодействие с пользователем, многопоточность и сетевое взаимодействие. Также была реализована внутренняя логика игры, включая проверку корректности расстановки, алгоритмы определения попадания, уничтожения кораблей и окончания игры.

Проект может быть улучшен путём добавления функции ручной расстановки кораблей пользователем, что повысит интерактивность и гибкость игрового процесса.

Список литературы

1. Иван Портянкин. Swing: Эффектные пользовательские интерфейсы, издание 2-ое. – Издательство “Лори”, 2011.
2. Лой Марк, Нимайер Патрик, Лук Дэниэл Л68 Программируем на Java. 5-е межд. изд. — СПб.: Питер, 2023. — 544 с.: ил. — (Серия «Бестселлеры O'Reilly»).
3. Шилдт, Герберт. Ш57 Java. Полное руководство, 12-е изд. : Пер. с англ. - СПб. : ООО “Диалектика”; 2023. - 1344 с.: ил. - Парал. тит. англ.

Приложение А. Листинг программы

Приложение А.1 Класс SeaBoy. Клиент.

```
package seaboy;
import java.awt.*;
import javax.swing.*;

public class SeaBoy {
    private final JButton buttonStart;
    private final JButton buttonPlace;
    private final JButton buttonFinish;
    private final Field fieldMy;
    private final Field fieldPartner;
    private JLabel labelStatus;

    SeaBoy(Game game) {
        Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
        int frameWidth = 950;
        int frameHeight = 670;

        JFrame f = new JFrame("Sea Boy");
        f.setLayout(new BorderLayout());
        f.setLocation((screenSize.width-frameWidth)/2, (screenSize.height-frameHeight)/2);
        f.setSize(frameWidth, frameHeight);
        f.setMinimumSize(new Dimension(frameWidth, frameHeight));
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Северная панель
        JPanel northPanel = new JPanel();
        northPanel.setBackground(new Color(153, 217, 234));
        northPanel.setPreferredSize(new Dimension(0, 75));
        northPanel.setLayout(new BorderLayout());

        JPanel leftNorthPanel = new JPanel();
        leftNorthPanel.setBackground(new Color(153, 217, 234));
        leftNorthPanel.setLayout(new FlowLayout(FlowLayout.LEFT, 15, 10));

        JPanel rightNorthPanel = new JPanel();
        rightNorthPanel.setBackground(new Color(153, 217, 234));
        rightNorthPanel.setLayout(new FlowLayout(FlowLayout.RIGHT, 15, 10));

        buttonStart = new JButton("Начать игру");
        buttonPlace = new JButton("Разместить");
        buttonFinish = new JButton("Завершить игру");
        JButton buttonAbout = new JButton("О программе");

        buttonStart.setPreferredSize(new Dimension(160, 40));
        buttonStart.setIcon(new ImageIcon("Icons/start(32).png"));
        buttonStart.setFocusPainted(false);
        buttonStart.setBackground(new Color(208, 226, 248));
        buttonStart.setBorder(BorderFactory.createLineBorder(new Color(105, 178, 197)));
        buttonStart.setFont(new Font("Comic Sans MS", Font.PLAIN, 14));

        buttonPlace.setPreferredSize(new Dimension(160, 40));
        buttonPlace.setIcon(new ImageIcon("Icons/place(32).png"));
        buttonPlace.setFocusPainted(false);
        buttonPlace.setBackground(new Color(208, 226, 248));
        buttonPlace.setBorder(BorderFactory.createLineBorder(new Color(105, 178, 197)));
        buttonPlace.setFont(new Font("Comic Sans MS", Font.PLAIN, 14));
```



```

buttonFinish.setPreferredSize(new Dimension(160, 40));
buttonFinish.setIcon(new ImageIcon("Icons/finish.png"));
buttonFinish.setFocusPainted(false);
buttonFinish.setBackground(new Color(208, 226, 248));
buttonFinish.setBorder(BorderFactory.createLineBorder(new Color(105, 178, 197)));
buttonFinish.setFont(new Font("Comic Sans MS", Font.PLAIN, 14));

buttonAbout.setPreferredSize(new Dimension(160, 40));
buttonAbout.setIcon(new ImageIcon("Icons/about(27).png"));
buttonAbout.setFocusPainted(false);
buttonAbout.setBackground(new Color(208, 226, 248));
buttonAbout.setBorder(BorderFactory.createLineBorder(new Color(105, 178, 197)));
buttonAbout.setFont(new Font("Comic Sans MS", Font.PLAIN, 14));

buttonStart.addActionListener(game);
buttonStart.setActionCommand("start");
buttonPlace.addActionListener(game);
buttonPlace.setActionCommand("place");
buttonFinish.addActionListener(game);
buttonFinish.setActionCommand("finish");
buttonFinish.setEnabled(false);
buttonAbout.addActionListener(game);
buttonAbout.setActionCommand("about");

leftNorthPanel.add(buttonStart);
leftNorthPanel.add(buttonPlace);
leftNorthPanel.add(buttonFinish);
rightNorthPanel.add(buttonAbout);
northPanel.add(leftNorthPanel, BorderLayout.WEST);
northPanel.add(rightNorthPanel, BorderLayout.EAST);

// Южная панель
JPanel southPanel = new JPanel();
southPanel.setLayout(new FlowLayout(FlowLayout.LEFT));
southPanel.setBackground(new Color(208, 226, 248));
southPanel.setPreferredSize(new Dimension(0, 65));
JLabel labelStatus = new JLabel();
labelStatus.setFont(new Font("Comic Sans MS", Font.PLAIN, 14));
southPanel.add(labelStatus);

// Основная панель
JPanel mainPanel = new JPanel();
mainPanel.setLayout(new BorderLayout(mainPanel, BorderLayout.X_AXIS));

// Западная панель
JPanel westPanel = new JPanel();
westPanel.setBackground(new Color(153, 217, 234));
westPanel.setPreferredSize(new Dimension(450, 0));
westPanel.setLayout(new BorderLayout());

JLabel labelNotMy = new JLabel("Мое поле", SwingConstants.CENTER);
labelNotMy.setFont(new Font("Comic Sans MS", Font.PLAIN, 18));
westPanel.add(labelNotMy, BorderLayout.NORTH);

fieldMy = new Field(game, true);
westPanel.add(fieldMy, BorderLayout.CENTER);

// Восточная панель
JPanel eastPanel = new JPanel();
eastPanel.setBackground(new Color(153, 217, 234));
eastPanel.setPreferredSize(new Dimension(450, 0));
eastPanel.setLayout(new BorderLayout());

```

```

JLabel labelMy = new JLabel("Поле противника", SwingConstants.CENTER);
labelMy.setFont(new Font("Comic Sans MS", Font.PLAIN, 18));
eastPanel.add(labelMy, BorderLayout.NORTH);

fieldPartner = new Field(game, false);
eastPanel.add(fieldPartner, BorderLayout.CENTER);

mainPanel.add(westPanel);
mainPanel.add(eastPanel);
f.add(northPanel, BorderLayout.NORTH);
f.add(southPanel, BorderLayout.SOUTH);
f.add(mainPanel, BorderLayout.CENTER);

f.setVisible(true);
}

public void onStateStarted(boolean stateStarted) {
    if(stateStarted) {
        buttonStart.setEnabled(false);
        buttonPlace.setEnabled(false);
        buttonFinish.setEnabled(true);
    }
    else {
        buttonStart.setEnabled(true);
        buttonPlace.setEnabled(true);
        buttonFinish.setEnabled(false);
    }
}

public void setStatusString(String message){
    labelStatus.setText(message);
}

public void setFieldCell(boolean my, int x, int y, Cell cell) {
    if(my)
        fieldMy.setCell(x, y, cell);
    else
        fieldPartner.setCell(x, y, cell);
}

public static void main(String[] args) {
    Client client = new Client(args[0]);
    Thread clientThread = new Thread(client, "clientThread");
    clientThread.setDaemon(true);
    clientThread.start();

    Partner partner = new Partner(client);
    client.setPartner(partner);

    Game game = new Game(partner);
    partner.setGame(game);

    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            SeaBoy seaBoy = new SeaBoy(game);
            game.setSeaBoy(seaBoy);
        }
    });
}
}

```

Приложение А.2 Класс Client. Клиент.

```
package seaboy;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.Socket;

public class Client implements Runnable {
    private Socket socket = null;
    private final String serverAddr;
    private final int serverPort = 60111;
    private Partner partner = null;

    public Client(String serverAddr) {
        this.serverAddr = serverAddr;
    }

    public void setPartner(Partner partner) {
        this.partner = partner;
    }

    @Override
    public void run() {
        System.out.println("client started");

        while(true) {
            try {
                socket = new Socket(serverAddr, serverPort);
                String addr = socket.getLocalAddress().getHostAddress();
                int port = socket.getLocalPort();
                System.out.println("client connected " + addr + " : " + port);
                readline();
            } catch (IOException ex) {
                System.out.println("run(): " + ex);
            }

            try {
                Thread.sleep(1000);
            } catch (InterruptedException ex) {
                System.out.println(ex);
                break;
            }
        }

        System.out.println("client finished");
    }

    /*
    private void read() {
        byte[] b = new byte[256];
        while(true) {
            try {
                InputStream in = socket.getInputStream();
                int len = in.read(b); //block
                if(len > 0) {
                    String s = new String(b, 0, len);
                    //System.out.println("read len = " + len + " : " + s);
                }
            } catch (IOException ex) {
                System.out.println(ex);
            }
        }
    }
    */
}
```

```

        if(partner != null) partner.onReceive(s);
    }
} catch (IOException ex) {
    System.out.println("read(): " + ex);
    break;
}
}
}
*/

private void readline() {
    try {
        InputStream in = socket.getInputStream();
        InputStreamReader r = new InputStreamReader(in);
        BufferedReader br = new BufferedReader(r);

        while(true) {
            String s = br.readLine();
            //System.out.println("read len = " + s.length() + " : " + s);
            if(partner != null) partner.onReceive(s);
        }
    } catch (IOException ex) {
        System.out.println("readline(): " + ex);
    }
}

public void write(byte[] b, int len) {
    try {
        OutputStream out = socket.getOutputStream();
        out.write(b, 0, len);
    } catch (IOException ex) {
        System.out.println("write(): " + ex);
    }
}

public void write(String s) {
    byte[] b = s.getBytes();
    write(b, b.length);
}

/*
public void close() {
    try {
        if(socket != null)
            socket.close();                //unblock
    } catch (IOException ex) {
        System.out.println(ex);
    }
}
*/
}

```

Приложение А.3 Класс Partner. Клиент.

```
package seaboy;

import java.util.Scanner;

public class Partner {
    private Client client = null;
    private Game game = null;

    public Partner(Client client) {
        this.client = client;
    }

    public void setGame(Game game) {
        this.game = game;
    }

    public void onReceive(String s) {
        Scanner scanner = new Scanner(s);
        String command = scanner.next();
        if(s.equals("START")) {
            //System.out.println("Partner requests to start the game");
            game.onReceiveStart();
        }
        else if(s.equals("FINISH")) {
            //System.out.println("Partner has finished the game");
            game.onReceiveFinish();
        }
        else if(command.equals("SHOT")) {
            int x = scanner.nextInt();
            int y = scanner.nextInt();
            //System.out.println(String.format("Partner made a shot at %d %d", x , y));
            game.onReceiveShot(x, y);
        }
        else if(command.equals("MISS")) {
            int x = scanner.nextInt();
            int y = scanner.nextInt();
            //System.out.println(String.format("Partner missed at %d %d", x , y));
            game.onReceiveMiss(x, y);
        }
        else if(command.equals("HIT")) {
            int x = scanner.nextInt();
            int y = scanner.nextInt();
            //System.out.println(String.format("Partner hit at %d %d", x , y));
            game.onReceiveHit(x, y);
        }
        else if(command.equals("KILL")) {
            int x = scanner.nextInt();
            int y = scanner.nextInt();
            game.onReceiveKill(x, y);
        }
    }

    public void sendStart() {
        client.write("START\n");
    }

    public void sendFinish() {
        client.write("FINISH\n");
    }
}
```

```
public void sendShot(int x, int y) {  
    client.write(String.format("SHOT %d %d\n", x, y));  
}  
  
public void sendMiss(int x, int y) {  
    client.write(String.format("MISS %d %d\n", x, y));  
}  
  
public void sendHit(int x, int y) {  
    client.write(String.format("HIT %d %d\n", x, y));  
}  
  
public void sendKill(int x, int y) {  
    client.write(String.format("KILL %d %d\n", x, y));  
}  
}
```

Приложение А.4 Класс Game. Клиент.

```
package seaboy;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Scanner;
import javax.swing.JOptionPane;

public class Game implements ActionListener {
    private final Partner partner;
    private SeaBoy seaBoy = null;
    private boolean stateStarted = false;
    private boolean sentStart = false;
    private boolean receivedStart = false;
    private final Cell[][] cellMy = new Cell[10][10];
    private final Cell[][] cellPartner = new Cell[10][10];
    private boolean placed = false;
    private boolean turn = false; //очередь хода

    public Game(Partner partner) {
        this.partner = partner;
        clearCellPartner();
        new Place(cellMy);
        placed = true;
    }

    public void setSeaBoy(SeaBoy seaBoy) {
        this.seaBoy = seaBoy;
        updateField(false);
        updateField(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        String s = e.getActionCommand();
        Scanner scanner = new Scanner(s);
        String command = scanner.next();
        if(s.equals("start")) {
            onButtonStart();
        }
        else if(s.equals("place")) {
            onButtonPlace();
        }
        else if(s.equals("finish")) {
            onButtonFinish();
        }
        else if(s.equals("about")) {
            onButtonAbout();
        }
        else if(command.equals("shot")) {
            int x = scanner.nextInt();
            int y = scanner.nextInt();
            onButtonShot(x, y);
        }
        else if(command.equals("my")) {
            int x = scanner.nextInt();
            int y = scanner.nextInt();
            onButtonMy(x, y);
        }
    }
}
```

```

public void onReceiveStart() {
    System.out.println("Partner requests to start the game");

    if(sentStart){
        setStateStarted(true);
        turn = false;
        seaBoy.setStatusString("Игра началась! Ход вашего друга");
    }
    else{
        receivedStart = true;
        seaBoy.setStatusString("Ваш друг хочет начать игру");
    }
}

public void onReceiveFinish() { // мы приняли
    if(!stateStarted) return;
    System.out.println("Partner has finished the game");
    seaBoy.setStatusString("Ваш друг завершил игру");
    setStateStarted(false);
    JOptionPane.showMessageDialog(null, new String[] {"Ваш друг завершил игру!"}, "Конец игры",
        JOptionPane.INFORMATION_MESSAGE);
}

public void onReceiveShot(int x, int y) {
    if(!stateStarted) return;
    System.out.println(String.format("Partner made a shot at %d %d", x , y));
    if(cellMy[x][y] == Cell.EMPTY) {
        seaBoy.setStatusString(
            String.format("Ваш друг промахнулся (%d, %d). Ваш ход", x, y));
        partner.sendMiss(x, y);
        cellMy[x][y] = Cell.MISS;
        seaBoy.setFieldCell(true, x, y, Cell.MISS);
        turn = true;
    }
    else if(cellMy[x][y] == Cell.SHIP) {
        seaBoy.setStatusString(
            String.format("Ваш друг попал (%d, %d). Ход вашего друга", x, y));
        partner.sendHit(x, y);
        cellMy[x][y] = Cell.HIT;
        seaBoy.setFieldCell(true, x, y, Cell.HIT);

        if(isKilled(x, y))
            setKilled(x, y);

        if(isOver(true)) {
            seaBoy.setStatusString("Вы проиграли!");
            setStateStarted(false);
            JOptionPane.showMessageDialog(null, new String[] {"Вы проиграли!"}, "Конец игры",
                JOptionPane.INFORMATION_MESSAGE);
        }
    }
}

public void onReceiveMiss(int x, int y) {
    System.out.println(String.format("You missed at %d %d", x , y));
    seaBoy.setStatusString(
        String.format("Вы промахнулись (%d, %d). Ход вашего друга", x, y));
    cellPartner[x][y] = Cell.MISS;
    seaBoy.setFieldCell(false, x, y, Cell.MISS);
    turn = false;
}

```



```

public void onReceiveHit(int x, int y) {
    System.out.println(String.format("You hit at %d %d", x , y));
    seaBoy.setStatusString(
        String.format("Вы попали! (%d, %d). Ваш ход", x, y));
    cellPartner[x][y] = Cell.HIT;
    seaBoy.setFieldCell(false, x, y, Cell.HIT);
}

public void onReceiveKill(int x, int y) {
    System.out.println(String.format("You killed at %d %d", x , y));
    cellPartner[x][y] = Cell.KILL;
    seaBoy.setFieldCell(false, x, y, Cell.KILL);

    if(isOver(false)) {
        seaBoy.setStatusString("Вы выиграли!");
        setStateStarted(false);
        JOptionPane.showMessageDialog(null, new String[] {"Вы выиграли!"}, "Конец игры",
            JOptionPane.INFORMATION_MESSAGE);
    }
}

private void onStart() {
    if(stateStarted) return;
    partner.sendStart();
    if(receivedStart){
        setStateStarted(true);
        turn = true;
        seaBoy.setStatusString("Игра началась! Ваш ход");
    }
    else{
        sentStart = true;
        seaBoy.setStatusString("Вы отправили запрос на начало игры");
    }
}

private void onButtonPlace() {
    if(stateStarted) return;
    new Place(cellMy);
    placed = true;
    updateField(true);
}

private void onButtonFinish() {
    if(!stateStarted) return;
    partner.sendFinish();
    setStateStarted(false);
    seaBoy.setStatusString("Игра завершена!");
}

private void onButtonAbout() {
    JOptionPane.showMessageDialog(null, new String[] {"Игра 'Морской бой'",
        "Выполнила: Зубриядова А.А. 22БВП1",
        "Руководитель: Юрова О.В.",
        "Кафедра: Вычислительная техника",
        "Пензенский государственный университет, 2025"}, "О программе",
        JOptionPane.INFORMATION_MESSAGE);
}

private void onButtonShot(int x, int y) {
    if(!stateStarted) return;
    if(!turn) {

```

```

        seaBoy.setStatusString("Ход вашего друга");
        return;
    }
    switch (cellPartner[x][y]) {
        case EMPTY -> {
            partner.sendShot(x, y);
        }
        case MISS -> {
            seaBoy.setStatusString(String.format("Вы давно промахнулись (%d, %d)", x, y));
        }
        case HIT, KILL -> {
            seaBoy.setStatusString(String.format("Вы давно попали! (%d, %d)", x, y));
        }
    }
}

private void onButtonMy(int x, int y) {
    if(!stateStarted) return;
    switch (cellMy[x][y]) {
        case EMPTY -> {
            seaBoy.setStatusString(String.format("Плещется водичка"));
        }
        case SHIP -> {
            seaBoy.setStatusString(String.format("Ваш корабль"));
        }
        case MISS -> {
            seaBoy.setStatusString(String.format("Дырка от бублика"));
        }
        case HIT, KILL -> {
            seaBoy.setStatusString(String.format("Ваш дырявый корабль"));
        }
    }
}

private void setStateStarted(boolean state) {
    if(stateStarted == state)
        return;
    stateStarted = state;
    System.out.println("stateStarted = " + stateStarted);

    seaBoy.onStateStarted(stateStarted);
    if(stateStarted) {
        if(!placed) {
            new Place(cellMy);
            updateField(true);
        }
    }
    else {
        clearCellPartner();
        updateField(false);
    }

    sentStart = false;
    receivedStart = false;
    placed = false;
}

private void updateField(boolean my) {
    for (int x = 0; x < 10; x++) {
        for (int y = 0; y < 10; y++) {
            if(my)
                seaBoy.setFieldCell(true, x, y, cellMy[x][y]);
        }
    }
}

```

```

        else
            seaBoy.setFieldCell(false, x, y, cellPartner[x][y]);
    }
}

private void clearCellPartner() {
    for (int x = 0; x < 10; x++) {
        for (int y = 0; y < 10; y++) {
            cellPartner[x][y] = Cell.EMPTY;
        }
    }
}

private boolean isOver(boolean my) {
    int c = 0;
    for (int x = 0; x < 10; x++) {
        for (int y = 0; y < 10; y++) {
            if(my) {
                if(cellMy[x][y] == Cell.KILL) c++;
            } else {
                if(cellPartner[x][y] == Cell.KILL) c++;
            }
        }
    }
    return (c == 20);
}

private boolean isKilled(int x, int y) {
    for (int i = x; i < 10; i++) {
        if((cellMy[i][y] == Cell.EMPTY) || (cellMy[i][y] == Cell.MISS)) break;
        else if(cellMy[i][y] == Cell.SHIP) return false;
    }
    for (int i = x; i >= 0; i--) {
        if((cellMy[i][y] == Cell.EMPTY) || (cellMy[i][y] == Cell.MISS)) break;
        else if(cellMy[i][y] == Cell.SHIP) return false;
    }
    for (int j = y; j < 10; j++) {
        if((cellMy[x][j] == Cell.EMPTY) || (cellMy[x][j] == Cell.MISS)) break;
        else if(cellMy[x][j] == Cell.SHIP) return false;
    }
    for (int j = y; j >= 0; j--) {
        if((cellMy[x][j] == Cell.EMPTY) || (cellMy[x][j] == Cell.MISS)) break;
        else if(cellMy[x][j] == Cell.SHIP) return false;
    }
    return true;
}

private void setKilled(int x, int y) {
    for (int i = x; i < 10; i++) {
        if(cellMy[i][y] == Cell.HIT) {
            partner.sendKill(i, y);
            cellMy[i][y] = Cell.KILL;
            seaBoy.setFieldCell(true, i, y, Cell.KILL);
        }
        else if(cellMy[i][y] == Cell.KILL) {}
        else break;
    }
    for (int i = x; i >= 0; i--) {
        if(cellMy[i][y] == Cell.HIT) {
            partner.sendKill(i, y);
            cellMy[i][y] = Cell.KILL;

```

```

        seaBoy.setFieldCell(true, i, y, Cell.KILL);
    }
    else if(cellMy[i][y] == Cell.KILL) {}
    else break;
}
for (int j = y; j < 10; j++) {
    if(cellMy[x][j] == Cell.HIT) {
        partner.sendKill(x, j);
        cellMy[x][j] = Cell.KILL;
        seaBoy.setFieldCell(true, x, j, Cell.KILL);
    }
    else if(cellMy[x][j] == Cell.KILL) {}
    else break;
}
for (int j = y; j >= 0; j--) {
    if(cellMy[x][j] == Cell.HIT) {
        partner.sendKill(x, j);
        cellMy[x][j] = Cell.KILL;
        seaBoy.setFieldCell(true, x, j, Cell.KILL);
    }
    else if(cellMy[x][j] == Cell.KILL) {}
    else break;
}
}
}

```

Приложение А.5 Класс Field. Клиент.

```
package seaboy;

import java.awt.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;

public class Field extends JPanel{
    private final JButton[][] button = new JButton[10][10];
    private final ImageIcon iconEmpty = new ImageIcon("Icons/empty.png");
    private final ImageIcon iconShip = new ImageIcon("Icons/ship.png");
    private final ImageIcon iconMiss = new ImageIcon("Icons/point.png");
    private final ImageIcon iconHit = new ImageIcon("Icons/cross.png");
    private final ImageIcon iconKill = new ImageIcon("Icons/kill.png");

    public Field(Game game, boolean my) {
        setBorder(new EmptyBorder(10, 10, 10, 10));
        JPanel gridPanel = new JPanel();
        setBackground(new Color(153, 217, 234));
        gridPanel.setLayout(new GridLayout(10, 10));
        //gridPanel.setPreferredSize(new Dimension(400, 400));

        for (int y = 0; y < 10; y++) {
            for (int x = 0; x < 10; x++) {
                button[x][y] = new JButton();
                button[x][y].setPreferredSize(new Dimension(40, 40));
                button[x][y].setIcon(iconEmpty);
                button[x][y].addActionListener(game);
                if(my)
                    button[x][y].setActionCommand(String.format("my %d %d", x, y));
                else
                    button[x][y].setActionCommand(String.format("shot %d %d", x, y));
                gridPanel.add(button[x][y]);
            }
        }
        add(gridPanel);
    }

    public void setCell(int x, int y, Cell cell) {
        switch (cell) {
            case EMPTY -> {
                button[x][y].setIcon(iconEmpty);
            }
            case SHIP -> {
                button[x][y].setIcon(iconShip);
            }
            case MISS -> {
                button[x][y].setIcon(iconMiss);
            }
            case HIT -> {
                button[x][y].setIcon(iconHit);
            }
            case KILL -> {
                button[x][y].setIcon(iconKill);
            }
        }
    }
}
```

Приложение А.6 Класс Place. Клиент.

```
package seaboy;

import java.util.Random;

public class Place {

    public Place(Cell[][] cell) {
        //Очистка поля
        for(int x = 0; x < 10; x++) {
            for(int y = 0; y < 10; y++) {
                cell[x][y] = Cell.EMPTY;
            }
        }

        for (int i = 0; i < 10; i++) {
            if (i == 0) ship(cell, 4);
            else if (i < 3) ship(cell, 3);
            else if (i < 6) ship(cell, 2);
            else ship(cell, 1);
        }
    }

    private void ship(Cell[][] cell, int shipLength) {
        Random random = new Random();
        while(true) {
            int x, y; // начальная клетка корабля

            // 0 - горизонтальное расположение (по оси x); 1 - вертикальное
            int location = random.nextInt(2);
            if (location == 1) {
                //System.out.println("Vertical");
                x = random.nextInt(10);
                y = random.nextInt(11 - shipLength);
            }
            else {
                //System.out.println("Horizontal");
                x = random.nextInt(11 - shipLength);
                y = random.nextInt(10);
            }

            boolean correctPlace = true;
            //System.out.println("x = " + x + " y = " + y );

            for (int i = -1; i <= shipLength; i++) {
                for (int j = -1; j <= 1; j++) {
                    int xx, yy;
                    if (location == 1) {
                        xx = x + j;
                        yy = y + i;
                    }
                    else {
                        xx = x + i;
                        yy = y + j;
                    }

                    if ((xx < 0) || (xx >= 10) || (yy < 0) || (yy >= 10))
                    {
                        continue;
                    }
                }
            }
        }
    }
}
```

```

        if (cell[xx][yy] != Cell.EMPTY) {
            correctPlace = false;
            break;
        }
    }
    if (!correctPlace) {
        //System.out.println("wrongPlace");
        break;
    }
}

if (correctPlace) {
    for (int i = 0; i < shipLength; i++) {

        if (location == 1) {
            cell[x][y+i] = Cell.SHIP;
            //System.out.println("Ship on x = " + x + ", y = " + (y + i));
        }
        else {
            cell[x+i][y] = Cell.SHIP;
            //System.out.println("Ship on x = " + (x+i) + ", y = " + y);
        }
    }
    break;
}
}
}
}

```

Приложение A.7 Cell. Клиент.

```
package seaboy;
```

```
public enum Cell {  
    EMPTY,  
    SHIP,  
    MISS,  
    HIT,  
    KILL  
}
```


Приложение А.8 Класс SeaServer. Сервер.

```
package seaserver;

import java.io.IOException;

public class SeaServer {

    public static void main(String[] args) {
        System.out.println("main started");

        Server server = new Server();
        Thread serverThread = new Thread(server, "serverThread");
        serverThread.setDaemon(true);
        serverThread.start();

        try {
            Thread.sleep(100);
        } catch (InterruptedException ex) {
        }

        try {
            System.out.println("press <enter> to exit");
            System.in.read();
        } catch (IOException ex) {
        }

        System.out.println("main finished");
    }
}
```

Приложение А.9 Класс Client. Сервер.

```
package seaserver;

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;

public class Client implements Runnable {
    private Socket socket = null;
    public boolean finished = false;

    public Client(Socket socket) {
        this.socket = socket;
        String addr = socket.getInetAddress().getHostAddress();
        int port = socket.getPort();
        System.out.println("client connected " + addr + " : " + port);
    }

    @Override
    public void run() {
        System.out.println("client started");
        read();
        System.out.println("client finished");
        finished = true;
    }

    private void read() {
        byte[] b = new byte[256];
        while(true) {
            try {
                InputStream in = socket.getInputStream();
                int len = in.read(b); //block
                if(len > 0) {
                    //write(b, len);
                    Server.onRead(this, b, len);
                }
            } catch (IOException ex) {
                System.out.println("read(): " + ex);
                break;
            }
        }
    }

    public void write(byte[] b, int len) {
        try {
            OutputStream out = socket.getOutputStream();
            out.write(b, 0, len);
        } catch (IOException ex) {
            System.out.println("write(): " + ex);
        }
    }
}
```

Приложение А.10 Класс Server. Сервер.

```
package seaserver;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Server implements Runnable {
    private ServerSocket serverSocket = null;
    private final int serverPort = 60111;
    private static final Client[] client = { null, null };

    @Override
    public void run() {
        System.out.println("server started");
        try {
            serverSocket = new ServerSocket(serverPort);
            String addr = serverSocket.getInetAddress().getHostAddress();
            int port = serverSocket.getLocalPort();
            System.out.println("server listen " + addr + " : " + port);

            while(true) {
                Socket socket = serverSocket.accept();          //block

                Client c = null;
                if((client[0] == null) || client[0].finished) {
                    c = new Client(socket);
                    client[0] = c;
                } else if((client[1] == null) || client[1].finished) {
                    c = new Client(socket);
                    client[1] = c;
                } else {
                    socket.close();
                }

                if(c != null) {
                    Thread clientThread = new Thread(c, "clientThread");
                    clientThread.setDaemon(true);
                    clientThread.start();
                }
            }
        } catch (IOException ex) {
            System.out.println("run(): " + ex);
        }

        System.out.println("server finished");
    }

    public static void onRead(Client c, byte[] b, int len) {
        if(c == client[0]) {
            if(client[1] != null)
                client[1].write(b, len);
        }
        else if(c == client[1]) {
            if(client[0] != null)
                client[0].write(b, len);
        }
    }
}
```

Приложение Б. UML – диаграммы

Приложение Б.1 UML – диаграмма последовательности

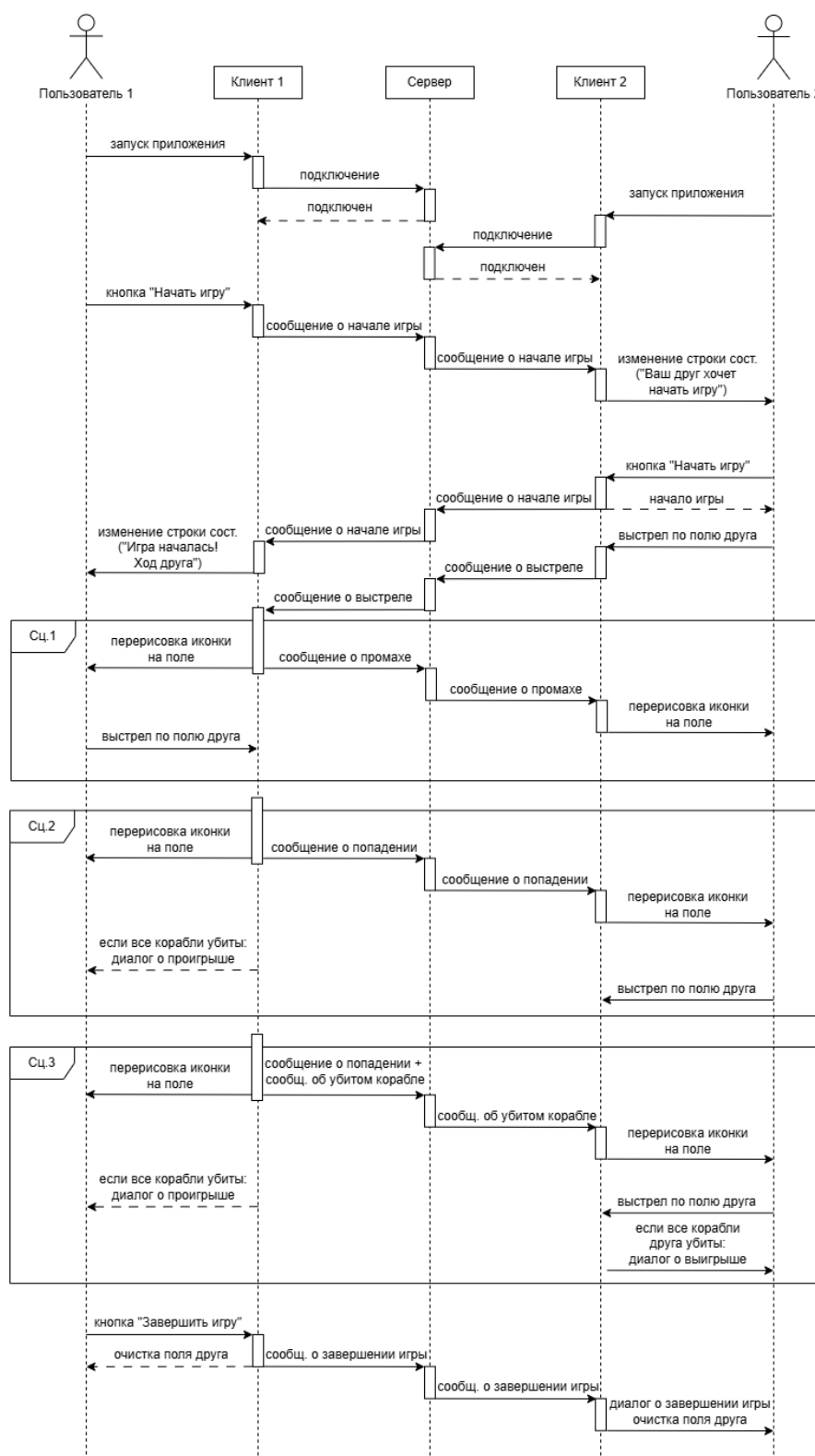


Рисунок 8 – Диаграмма последовательности

Приложение Б.2 UML – диаграммы классов

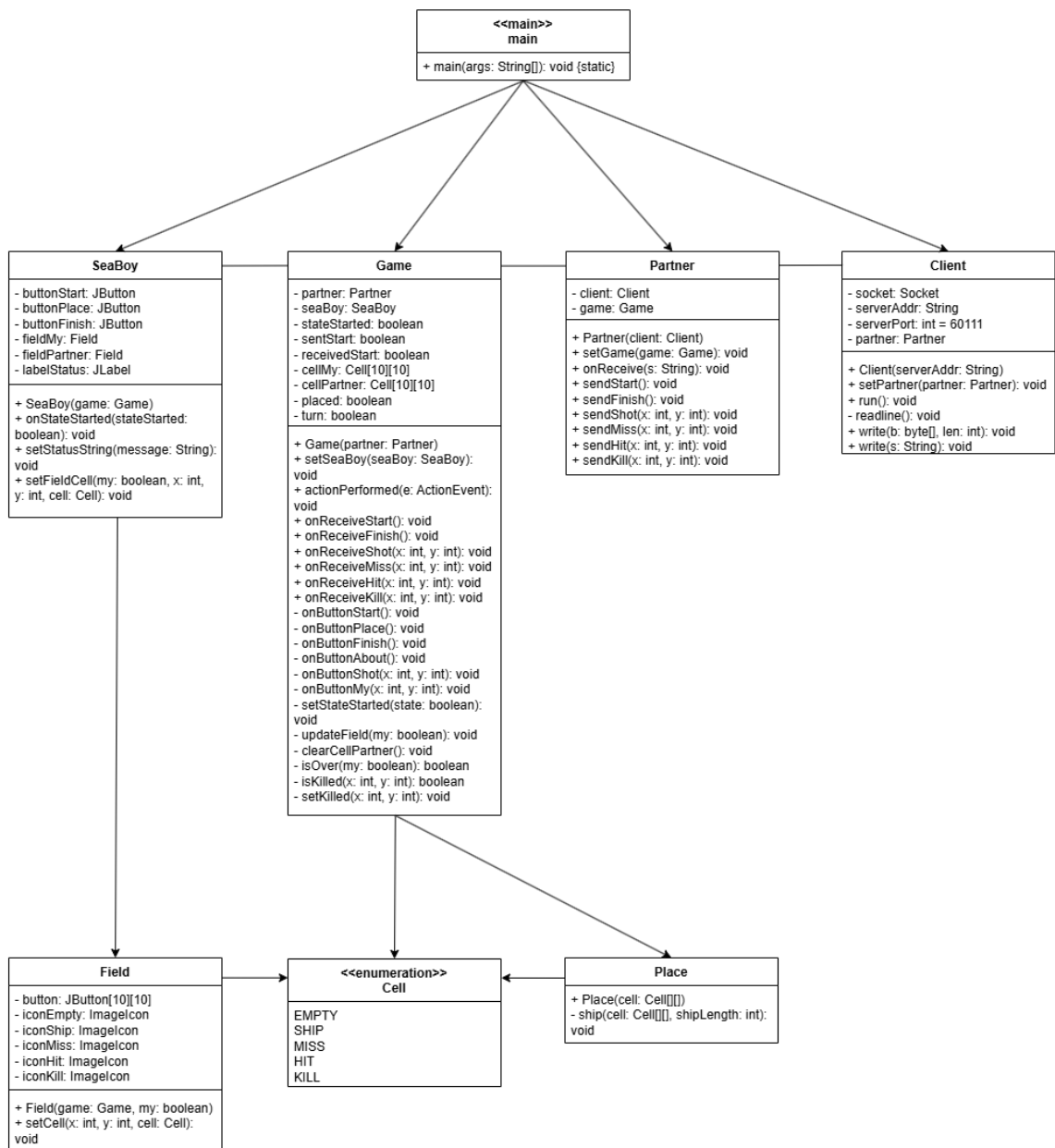


Рисунок 9 – Диаграмма классов программы-клиента

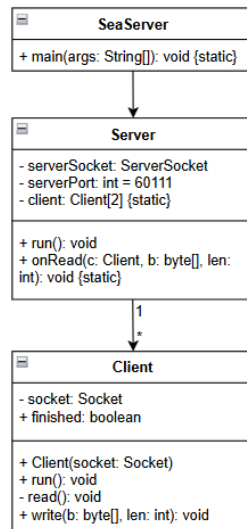


Рисунок 10 – Диаграмма классов программы-сервера

Приложение Б.3 UML – диаграмма деятельности

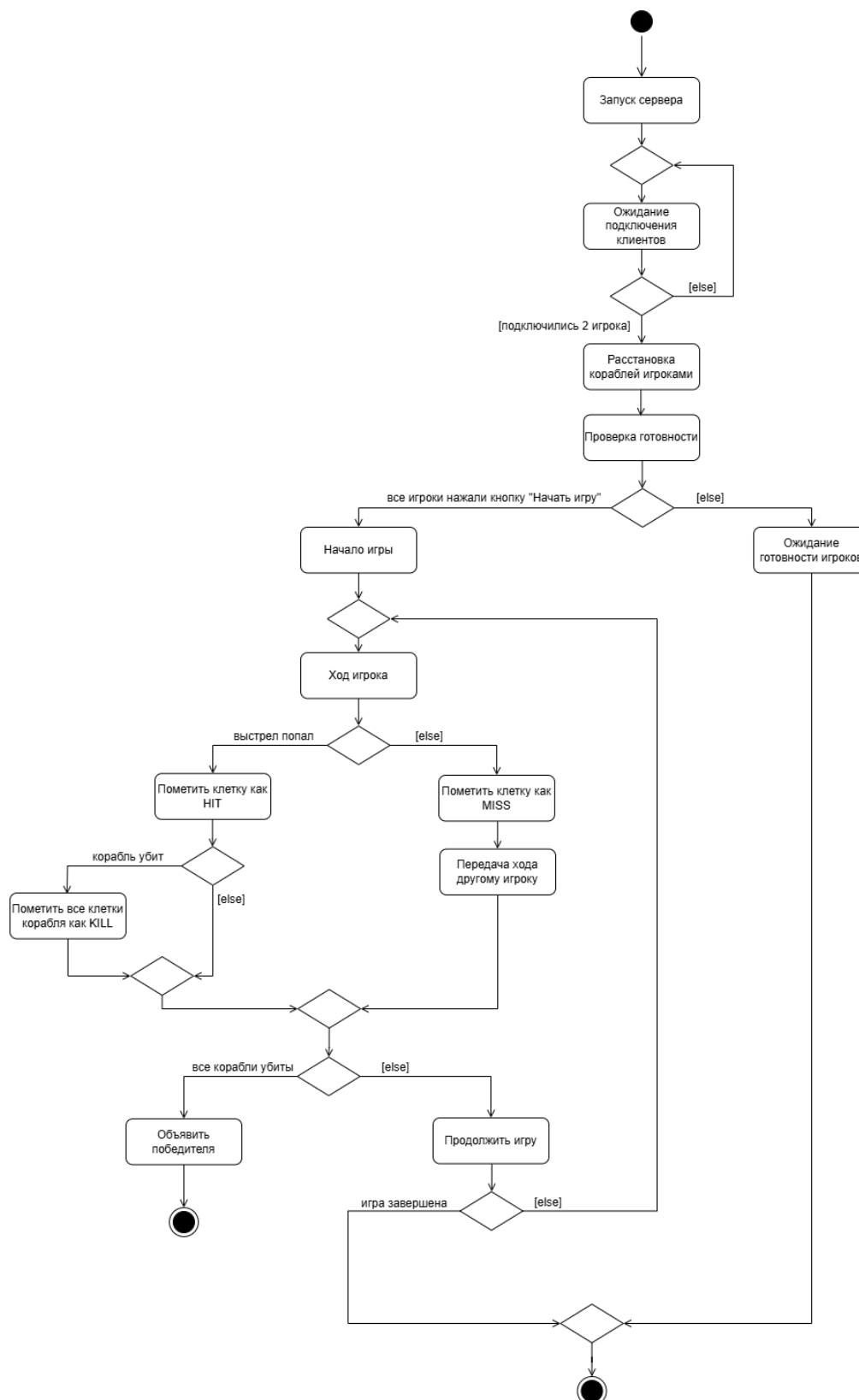


Рисунок 11 – Диаграмма деятельности

Приложение Б.4 UML – диаграмма вариантов использования

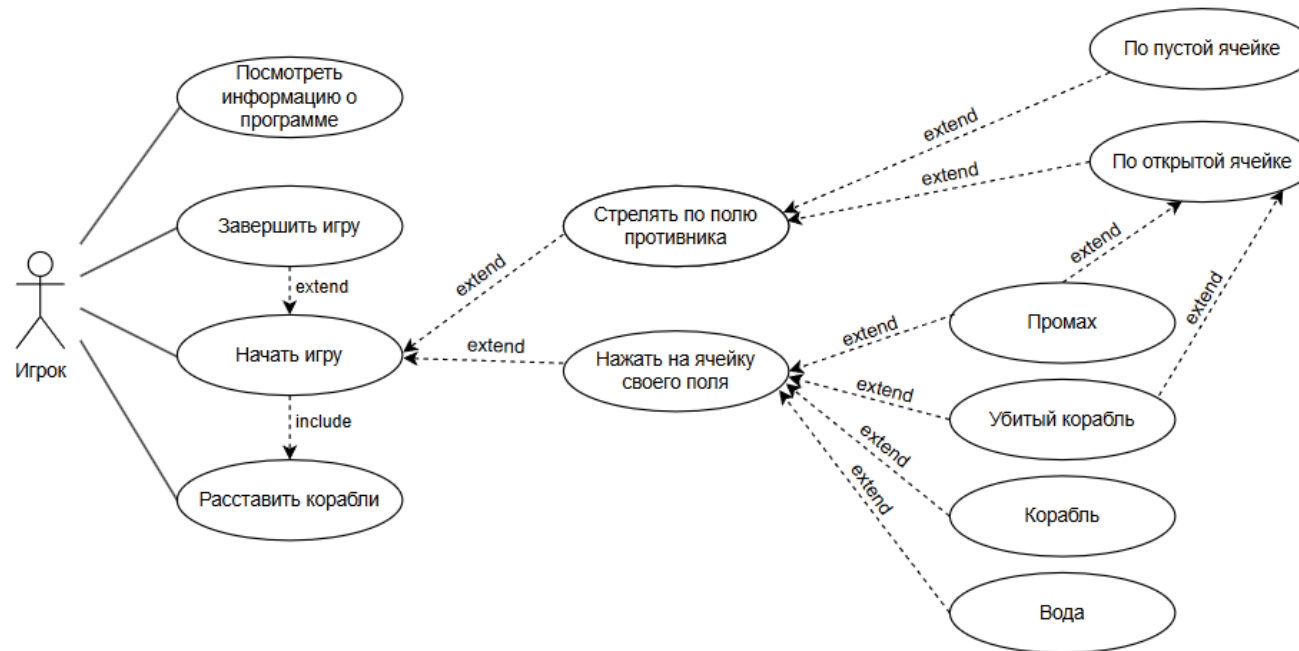


Рисунок 12 – Диаграмма вариантов использования

Приложение Б.5 UML – диаграмма развертывания



Рисунок 13 – Диаграмма развертывания